

StarCDN：将内容分发网络迁移至太空

报告人：

胡博瑄 (计算机试验班 2201. 学号: 2223410945)

论文作者：

William X. Zheng, Aryan Taneja, Maleeha Masood.单位: University of Illinois Urbana-Champaign

Anirudh Sabnis.单位: Akamai Technologies

Ramesh K. Sitaraman.单位: University of Massachusetts Amherst, Akamai Technologies

Deepak Vasisht.单位: University of Illinois Urbana-Champaign

摘要

低地球轨道卫星网络（如 Starlink）为全球数百万用户提供互联网接入服务。近期研究表明，现有的网络基础设施（如内容分发网络，CDN）并不适用于卫星网络架构。传统地面 CDN 会降低卫星网络用户的访问性能，且无法缓解地面-卫星链路上的拥塞问题。为此，我们设计了 StarCDN——一种全新的 CDN 架构，通过在太空中缓存内容来提升用户体验并降低地面-卫星链路的带宽消耗。设计 StarCDN 的核心挑战在于卫星的轨道运动，这使得每颗卫星的覆盖区域在数分钟内快速变化，从而服务于截然不同的地理区域（例如从美国切换到欧洲）。为应对这一挑战，我们引入了专为 LEO 卫星网络定制的一致性哈希和中继取回机制。我们的设计使缓存内容能够沿轨道运动的反方向流动，从而抵消卫星运动带来的影响。我们基于 Akamai 的真实流量数据，在多种基线方案下对 StarCDN 进行了评估。实验结果表明，StarCDN 可将地面-卫星链路的带宽利用率降低 80%，并将用户感知延迟改善 2.5 倍。此外，我们还开源了一个合成流量生成器 SpaceGEN，用于支持卫星 CDN 的真实仿真。

引言

通过大规模星座在低地球轨道运行的卫星网络正在迅速发展。Starlink 作为领先的 LEO 卫星网络提供商，已在 100 多个国家拥有超过 400 万订阅用户。目前其运营着超过 7000 颗卫星，并计划扩展至 40000 颗。类似地，OneWeb 和 Amazon Kuiper 等公司也提供或计划提供类似的 LEO 卫星网络服务。LEO 卫星网络（LSN）正日益被用于向全球用户分发内容。我们的工作聚焦于提升 LSN 内容分发的性能并降低其运营成本。

内容分发网络（CDN）：CDN 于四分之一世纪前被发明，旨在提升地面互联网上网站、媒体和下载内容的分发性能并降低成本。CDN 在全球部署数十万台“边缘”服务器来缓存内容，并从“接近”用户的位置提供服务。CDN 使内容能以更低的延迟和更高的性能提供服务，因为内容经过从邻近边缘服务器到用户的更短网络路径。CDN 还能降低成本和网络利用率，因为内容只需从源服务器下载一次到 CDN 边缘服务器，随后可多次向用户分发，节省了上游 WAN 带宽（即中间段带宽，midgress）。CDN 已成为现代互联网基础设施的核心组件，服务着近 75% 的全球互联网流量。

现有技术的不足：将传统（地面）CDN 技术与 LSN 结合使用存在多项不足。近期研究表明，在 Starlink 上使用传统地面 CDN 会降低用户的体验质量（QoE）。这种性能下降源于用户流量在 Starlink 中通过弯管（bent-pipe）架构传输——用户连接至卫星，卫星再连接至最近的地面站。地面站随后连接至地面 CDN 的边缘服务器，导致延迟增加超过 100 毫秒。此外，少部分网络流量还可能通过星间链路（ISL）传输——ISL 拥有相较于地面-卫星链路（20Gbps）更为充裕的带宽（100Gbps）。

我们的研究焦点：驱动我们工作的核心问题是：CDN 技术能否用于提升 LSN 用户的性能、改善网络利用率并降低内容分发的运营成本。我们的主要方法是探索能否在 LEO 卫星上部署边缘服务器系统来缓存内容并为（地面）用户提供服务。然而，实现这一方法的重大挑战在于边缘服务器处于运动状态——卫星每 90 分钟绕地球一周。与传统 CDN 中边缘服务器固定不同，在我们的场景中，与边缘服务器相邻的用户及其内容访问模式都在分钟级别动态变化。在 LSN 的新场景下重新思考

CDN 的边缘集群架构，特别是其内容放置、请求路由和缓存机制，是我们工作的核心贡献。

我们的方法：我们提出了一种名为 StarCDN 的天基内容分发网络，专为 LSN 设计。StarCDN 利用卫星新兴的计算能力部署边缘服务器，在太空中缓存内容，从而降低用户的访问延迟并提升体验质量。同时，StarCDN 减少了卫星网络所需的上行带宽，使稀缺的频谱资源可被重新分配给下行需求。在设计 StarCDN 的过程中，我们解决了三个关键挑战：

(i) 多卫星冗余：为在长距离上提供最优服务，LSN 采用由数千颗卫星组成的密集部署。因此，在任意时刻，一个 Starlink 用户可连接 10 颗以上的卫星。由于卫星轨道运动，这组可见卫星是动态的，可在数分钟内发生变化。由于用户可连接任何可见卫星，因此所请求的内容必须缓存在所有可见卫星上，才能可靠地从缓存中提供服务。这导致高缓存未命中率和卫星上宝贵存储空间的浪费。为此，我们提出了一种一致性哈希方案，将每个对象哈希到 K （例如 $K=4$ ）个桶中，每个桶映射到不同的卫星。当卫星收到属于其桶的数据请求时，可直接响应（缓存命中）或从地面请求数据（缓存未命中）。当卫星收到属于不同桶的请求时，它通过 ISL 链路将请求转发至拥有该桶的邻近卫星。我们将桶映射到网格模式的卫星上，使每个桶最多在 $2\lceil\frac{\sqrt{K}}{2}\rceil$ 跳内可达。我们的一致性哈希方案优化了太空中的存储利用率，同时最小化了请求的额外延迟。

(ii) 轨道运动：在地面 CDN 中，对象缓存在物理上接近用户的服务器中。缓存决策依赖于服务区域的本地流行度统计。然而，LEO 卫星以约 8 公里/秒的速度绕地球运行，对给定位置的服务时间不超过十分钟。服务美国用户的卫星在数分钟内可能转而服务欧洲用户。因此，访问模式、流行度统计和缓存内容迅速过时，导致低命中率。我们的分析表明，部署在卫星上的简单 LRU 缓存由于轨道运动，命中率将低于 60%。为应对轨道运动的影响，StarCDN 部署了中继取回技术——卫星可在缓存未命中时，通过 ISL 链路将请求中继到具有相同桶 ID 的邻近卫星。这使得 StarCDN 能从刚刚服务过某区域的前序卫星的缓存中获益。实际上，这使得缓存内容能沿轨道运动的反方向流动，从而抵消轨道运动的影响。

(iii) 一种新型的、公开可用的合成流量生成器，用于捕获全球分布的 LSN 用户内容访问模式：评估天基 CDN 设计面临的挑战在于需要全球分布的流量轨迹，以捕

获世界各地用户的内容访问行为。传统 CDN 设计通常使用来自一个（或少数）位置的流量轨迹进行评估。然而，由于卫星环绕全球运行，要有效评估 StarCDN，我们需要来自地球上多个位置、长时间跨度的流量轨迹。为此，我们首先从 Akamai CDN 收集了来自全球九个位置的有限真实流量轨迹。随后，我们设计了一个新型合成流量生成器 SpaceGEN，利用 footprint descriptors 捕获单一位置内内容访问模式的时间变化（如对象访问频率）以及跨位置的地理变化（如跨位置的内容访问重叠）。SpaceGEN 还可为 CDN 承载的主要流量类别（如 Web、视频和软件下载）生成合成流量。我们的流量生成器支持对 StarCDN 及其他基线方案进行真实的长期评估。

贡献总结：

- StarCDN 是首个天基 CDN，旨在提升 LSN 用户的内容访问体验，同时优化地面-卫星网络的利用率 (§3)。
- 我们设计并评估了一种新型的、面向 LSN 的一致性哈希方案以减少冗余并提升缓存性能 (§3.2)。同时设计并评估了一种中继取回方案以对抗卫星轨道运动的影响 (§3.3)。
- StarCDN 基于来自全球分布的 LSN 用户的真实内容访问轨迹进行评估。这些合成轨迹由我们的新型流量生成器 SpaceGEN 生成，其产生的合成轨迹与视频、Web 和下载等不同流量类别的实际生产轨迹高度相似 (§4)。据我们所知，SpaceGEN 是首个同时融入用户内容访问模式的时间变化和地理变化的缓存仿真流量生成器。为推动该领域的研究，我们已在 GitHub 上开源了 StarCDN 仿真框架和 SpaceGEN 流量生成器。
- 我们使用 Microsoft CosmicBeats 仿真器模拟了实际卫星上的 CDN 边缘服务器，同时模拟了卫星轨道运动和视场。我们对 Starlink 星座的 1170 颗卫星进行了仿真。实验表明，StarCDN 可将太空中的缓存命中率从 60%提升至 75%，将卫星网络上行带宽利用率降低高达 80%，并将用户感知延迟改善 2.5 倍。

背景

我们简要介绍 LSN 和 CDN 的背景知识。

2.1 LEO 卫星网络

基于 LEO 卫星的互联网服务在过去十年获得了广泛应用，Starlink、OneWeb 和 Kuiper 等服务的出现推动了这一趋势。Starlink 是最成熟的 LEO 网络服务提供商，在轨运行超过 7000 颗卫星。

轨道运动：LEO 卫星在约 550 公里的高度绕地球运行（相比传统地球同步卫星的 36000 公里）。因此，LEO 卫星的传播延迟显著降低，能更好地支持现代互联网应用。LEO 卫星还因靠近地球而拥有更好的链路带宽预算（从而获得更高的吞吐量）。然而，LEO 卫星必须保持高速运行以维持其轨道。由此，LEO 卫星约每 90 分钟绕地球一周。由于这种快速轨道运动，用户到卫星的连接时间最多只有几分钟，导致频繁的用户-卫星链路“切换”。

网络路由：Starlink 卫星配备有用于地面-卫星链路的无线电天线和用于星间链路（ISL）的光学收发器。常见的通信模式是弯管（bent pipe）模型——用户终端连接至 Starlink 卫星，卫星再连接至 Starlink 地面站。地面站通过地面网络将数据包转发，并通过互联网交换点（IXP）连接至互联网的其余部分。这一流程如图 1 所示。

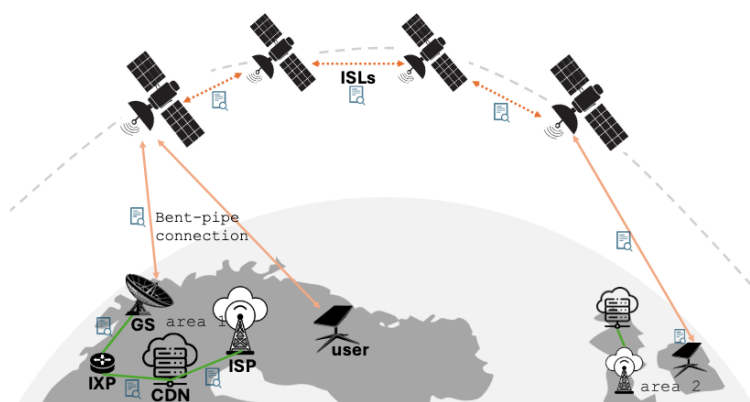


Figure 1: Satellite network users often experience higher latencies compared to terrestrial users when accessing content due to the bent-pipe architecture where a user connects to a CDN's edge server via one more satellites and a ground station (GS), traversing ground-to-satellite links and perhaps also inter-satellite links (ISLs).

ISL 最近已在 Starlink 中引入，常用于用户附近没有地面站的场景（如非洲许多国家）。卫星使用 ISL 将流量路由到其他卫星，再下行到地面站。Starlink 卫星通常支持四条 ISL：两条轨道内链路（连接同一轨道中的前后卫星）和两条轨道间链路（连接平行轨道中的相邻卫星）。各链路类型的传播延迟和带宽如表 1 所示。

	Avg Delay(ms)	Std Delay(ms)	Min Delay(ms)	Bandwidth (Gbps)
Intra-orbit ISL	8.03	0.376	4.76	100
Inter-orbit ISL	2.15	0.492	1.32	100
GSL	2.94	1.01	1.82	20

Table 1: *Propagation delay and bandwidth of Starlink links.*

2.2 内容分发网络

内容分发网络（CDN）是一个可能由数十万台边缘服务器组成的大规模分布式系统，部署在全球数千个位置。边缘服务器以集群形式部署，每个集群位于一个数据中心或托管设施内。CDN 边缘服务器可代表数千个内容提供商缓存并向用户分发内容。虽然源服务器存储所有对象，但由于缓存空间有限，CDN 边缘服务器无法缓存所有对象。当用户请求内容（如网页或视频）时，该请求被路由到邻近的 CDN 边缘服务器。如果该服务器拥有所请求的内容，则称为缓存命中（cache hit），用户获得该内容。如果边缘服务器没有请求的内容，则称为缓存未命中（cache miss），内容从源服务器通过 WAN 获取并提供给用户。当边缘服务器缓存满时，CDN 提供商使用驱逐策略决定应从缓存中移除哪些内容。最常用的策略是最近最少使用（LRU）策略，其不同变体因简单性和有效性被广泛部署于商业 CDN 中。

传统 CDN 基础设施在地理上是静态的，依赖于将边缘服务器部署在靠近用户的位置，并将用户请求路由到能够提供内容的邻近边缘服务器。缓存命中率是需要最大化的关键指标，因为每次缓存未命中都会增加用户体验到的延迟和从源服务器获取“未命中”内容的上游 WAN 带宽。命中率有两种衡量方式——请求命中率（request hit rate）是缓存命中请求所占的比例，而字节命中率（byte hit rate）是缓存命中请求所服务字节数的比例。

2.3 太空计算与存储的可行性

太空中的 CDN 边缘服务器需要卫星提供存储、计算和电力资源。我们讨论在卫星上提供这些资源的可行性。近十年来，在太空中放置计算和存储能力方面已取得重大进展。在轨计算探索了为卫星增加边缘计算能力的可行性，这与 CDN 集成相关。相关工作论证了在 Starlink 卫星上部署最高 2TB 存储的高端服务器在功率、重量和体积方面的可行性。更新型的服务器可以在相同要求下提供更多的存储和计算能力，表明对体积和重量的需求将随技术进步而降低。一些任务已经发射了具有太空计算能力的卫星。例如，Planet 和欧洲航天局已展示了在小型卫星上使用边缘计算设备（如 NVIDIA Jetson）运行机器学习模型的能力。

StarCDN 系统架构

我们提出一种新型天基内容分发网络 StarCDN，专为 LSN 设计。StarCDN 在卫星中部署 CDN 边缘服务器来缓存内容，同时利用 ISL 链路从邻居按需获取内容。在设计 StarCDN 时，我们追求以下目标：

- 降低延迟：卫星网络用户在访问内容时应体验到与地面网络用户相似的延迟。
- 降低上行带宽利用率：目前，向用户提供的所有内容都必须通过上行链路从地面站发送至卫星。我们旨在通过减少从地面获取内容的需求来降低上行利用率。这一目标源于卫星服务需求的激增——Starlink 用户基数从约 100 万增长至全球超过 400 万，导致上行和下行带宽竞争加剧。降低上行利用率将释放可被重新分配给更多用户的上行带宽。
- 与现有网络架构兼容：我们旨在保持与现有卫星网络架构的兼容性，即仔细建模 GSL、ISL、星座大小、卫星轨道和地面站，使用 Starlink 网络的公开信息。

3.1 朴素设计及其面临的挑战

为阐明 StarCDN 的设计决策，我们首先考虑一种朴素的卫星 CDN 设计。设想 Starlink 星座中每颗卫星都配备一台可缓存内容的边缘服务器，每个缓存独立采用 LRU 驱逐策略。我们分析这种设置面临的卫星网络独有挑战。

3.1.1 挑战一：轨道运动导致的动态访问模式

一颗 LEO 卫星每 90 分钟绕地球一周。由于地球每 24 小时自转一周（与 LEO 卫星不同步），LEO 卫星在每个轨道周期覆盖地球的不同区域（参见图 3 的示意）。这意味着每个缓存随时间服务不同地理区域的用户。我们注意到，这与地面 CDN——服务器固定且通常服务于邻近区域用户——是截然不同的场景。

为量化轨道运动的影响，我们研究了来自欧洲不同官方语言国家间对象和内容访问流量的重叠度（表 2）。我们注意到，不同语言即使在单一大洲内也造成了显著的流量差异。这些多样化的地区可在数分钟内被同一卫星遍历和服务。

此外，我们在图 2 中绘制了与纽约地理距离相关的唯一对象和唯一流量的重叠度。分析揭示了几个有趣的发现。首先，对于距纽约较近的地区（<3000 公里），对象重叠度约为 55%。其次，跨洲（>3000 公里）的重叠度在唯一对象和流量方面都相当低。当一颗卫星积累了足够的流量来建立缓存时，其轨道运动已将它拉到了一个不同的区域。

要点：LEO 卫星的快速轨道运动使其在数分钟内覆盖不同的地理区域。这些区域需要不同的缓存内容。

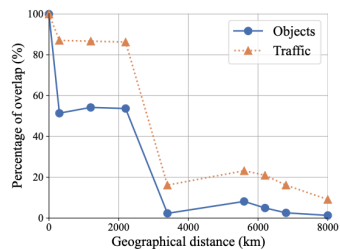


Figure 2: Percent of objects and traffic accessed by users in New York that are also accessed by users at a different location. The overlap decreases with the geographic distance of the location from New York.

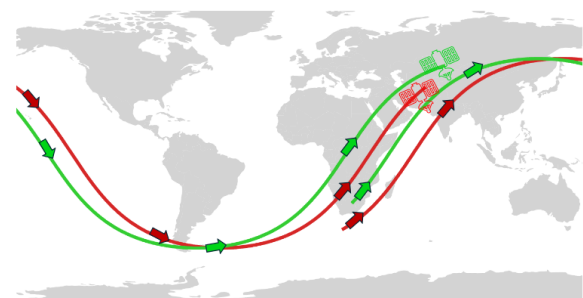


Figure 3: Trajectory of two satellites, three parallel orbits away.

	Britain	Germany	Turkey
Britain	100%	11%(49%)	2%(15%)
Germany	16%(45%)	100%	4%(31%)
Turkey	23%(37%)	34%(72%)	100%

Table 2: Percent of objects (traffic) accessed by users in the first country (row) that are also accessed in the second country (column). The content overlap is low since users in these European countries speak different languages and seldom access the same content.

3.1.2 动态客户端-服务器关系

上述轨道运动的一个推论是，卫星网络中的客户端-服务器关系不断演变。在地面 CDN 中，通过映射系统（DNS 系统或 IP 任播）将客户端请求路由到邻近的边缘集群，集群内的负载均衡器将客户端分配到边缘服务器。由于服务器固定且客户端通常位于同一地理区域，客户端的服务器分配通常是稳定的。

然而，LEO 卫星在低轨道绕地球运行，每颗卫星只能覆盖地球的一小部分区域。在任意时刻，一个 Starlink 客户端视野中通常有 10 颗以上的卫星，Starlink 调度器负责安排客户端到卫星的连接。我们将调度器分配给客户端的卫星称为首接触卫星，该映射变化迅速。在任何 LEO 网络中，客户端-卫星映射无法维持超过几分钟。这意味着服务同一区域的多颗卫星可能需要缓存和提供相同的对象，导致冗余，降低了缓存存储效率。

考虑图 4 的示例。user-1 和 user-2 分别被调度到卫星 D 和 B。user-1 发起请求，由卫星 D 检查本地缓存并提供。如果 user-2 请求相同内容，卫星 B 将获取并缓存相同内容，而未意识到该请求可通过邻近卫星 D 的缓存通过 ISL 链路满足。因此，两颗卫星中相同内容的冗余存储降低了整体缓存存储效率。

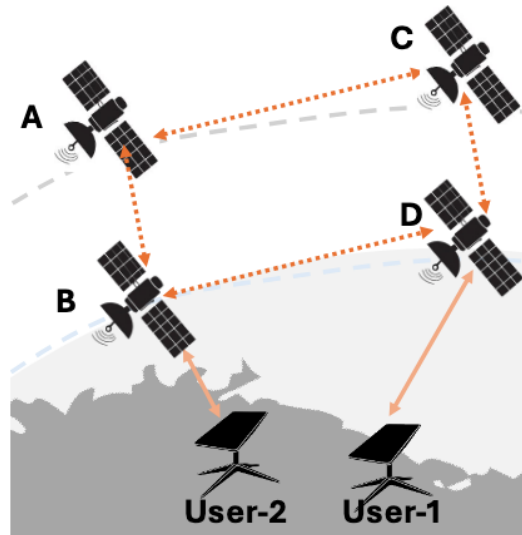


Figure 4: Two users with four satellites in view. Each user is connected to one of these satellites by Starlink’s satellite link scheduler.

要点：动态客户端-服务器映射可能导致相同内容在多个缓存中冗余存储，降低整体缓存存储效率。此外，缓存未命中时可能导致更大延迟，因为需要额外的地面-卫星通信。

3.2 StarCDN：通过一致性哈希减少冗余缓存

为应对动态客户端-服务器关系，我们提出了一种一致性哈希方案，用于在卫星间划分缓存内容。一致性哈希广泛应用于生产存储系统以提供负载均衡和容错能力。从概念上讲，一致性哈希将服务器和对象都哈希到一个单位圆上，每个对象映射到圆上顺时针方向的下一个服务器。在地面网络中，一致性哈希在集群内位置相近的服务器间进行。

我们无法直接将一致性哈希应用于卫星场景。具体而言，LEO 卫星通常较小，在有限的功率、体积和热约束下无法同时运行多个服务器。此外，即使将对象映射到卫星上的不同服务器，也无法解决前述的冗余缓存问题。

StarCDN 提出了一种卫星特定的一致性哈希变体。类似于标准一致性缓存，我们将对象划分到 K 个不相交的桶中。随后，每个桶映射到一颗不同的卫星。但如何确定哪个桶映射到哪颗卫星？形式上，对于任意星座拓扑，该问题可建模为图着色问题，需考虑 ISL 的存在及从不同卫星获取内容的延迟约束。

Starlink 拓扑呈网格模式，简化了映射问题。星间链路自然形成卫星的网格拓扑结构。每颗卫星通常连接前、后、左、右四颗卫星。我们将 K 个桶映射到该网格上，形成一个重复的 $\sqrt{k} \times \sqrt{k}$ 模式。例如，可以将 9 个桶映射到 3×3 模式中。

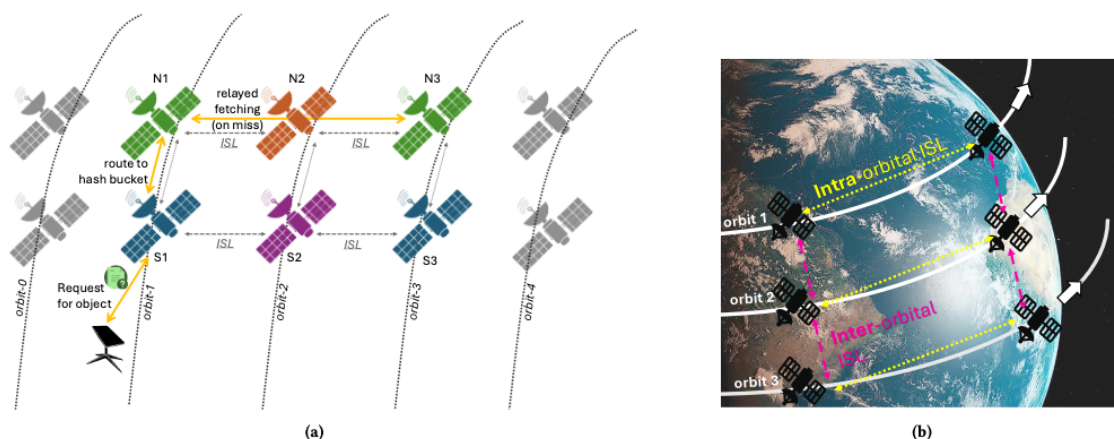


Figure 5: (a) Consistent hashing routing and relayed fetching in StarCDN. (b) Orbital motion and ISLs of Starlink satellites.

图 5a 展示了 $K=4$ 的示例，其中四个桶的内容分别存储在每个 2×2 网格中。需注意的是，我们无法控制用户连接到哪颗卫星，这由 Starlink 的卫星链路调度器决定。然而，当用户请求一个对象时，其首接触卫星（如图 5a 中的 S1）可以检查

该对象是否在其指定桶中。如果是，则直接从本地缓存中响应（缓存命中）或从地面请求数据（缓存未命中）。如果不是，则计算到拥有该对象桶的卫星（如 N1）的最短路径，并沿该路径转发请求。如果 N1 拥有该对象（缓存命中），则将对象转发回首接触卫星（S1）。如果 N1 没有（缓存未命中），则可从地面请求数据，存入其缓存，并转发给首接触卫星（S1）。

该方案具有多项优势。首先，它提高了缓存存储效率，因为每个缓存只需存储 $(1/K)^{th}$ 的对象，使更多对象可被缓存，从而提高缓存命中率。其次，它减少了冗余，因为每个对象只由其分配桶的服务器存储。第三，尽管查询邻近卫星的对象有额外延迟，但所有桶最多在 $2\lceil\frac{\sqrt{K}}{2}\rceil$ 跳内可从首接触卫星到达。最后，该一致性哈希方案兼容任何缓存替换策略，包括 LRU、LFU、SIEVE 等。

如何选择 K 值？K 的选择由三个因素驱动。首先，较小的 K 会增加缓存未命中率，因为分配给每个缓存的内容比例更大，内容冗余更多。其次，较大的 K 会增加访问对象的延迟。第三，星座大小、轨道设计和轨道运动存在约束。我们发现 K=4 和 K=9 通常与 Starlink 星座兼容。我们在§5.3 中评估了这一权衡。

3.3 StarCDN：通过中继取回对抗轨道运动

接下来，我们讨论 StarCDN 如何应对访问模式动态偏移的挑战。这种动态访问模式偏移的根本原因是卫星的轨道运动。与传统 CDN 中边缘服务器固定在单一位置并服务邻近用户不同，LEO 卫星相对于地面快速运动。直觉上，我们的目标是创建一个沿轨道运动反方向流动的缓存内容流。这使得特定位置的用户所访问的缓存内容能保持在该位置上方，而缓存该内容的卫星因轨道运动不断改变。我们称这一技术为中继取回（relayed fetch），即允许卫星从映射到相同桶的邻近卫星请求对象。例如，在图 5a 中，N1 可在缓存未命中时从其右侧的下一个最近同色邻居 N3 请求内容。此类中继仅在缓存未命中时发起。

中继取回的一个考量是延迟开销。每次单向跨轨道跳的延迟至少为 2ms，而轨道内跳约需 8ms。鉴于轨道间链路的更大距离，StarCDN 仅从轨道间邻居取回数据，避免轨道内邻居，以减轻缓存未命中带来的高延迟惩罚。

为了直观理解星间链路及取回的好处，我们在图 3 中可视化了两颗相距三个平行轨道的卫星的轨迹。可以观察到，红色卫星的轨迹与绿色卫星（红色卫星的西侧轨

道间邻居) 在前一个时段走过的路径非常相似。这意味着卫星的西侧邻居拥有我们希望利用的请求历史足迹。

最后, 我们还允许卫星从其东侧轨道间邻居取回内容 (例如 N1 和 N3 之间), 因为这与从西侧邻居取回具有相同的延迟惩罚。我们在 StarCDN 中保持这些链路是双向。

为何不采用主动预取? 一种替代策略是在进入人口密集区域时从前序卫星主动预取热门内容。它可以创建类似的反向内容流。然而, 预取存在获取过期内容 (已不再被客户端请求) 的风险。我们发现这种策略在命中率方面不如中继取回高效。中继取回虽然在首次请求时产生额外延迟, 但对象被取回后即可存入本地缓存。相比之下, 如果主动预取的内容未被使用, 则浪费了接收端的缓存空间、传输数据的电力以及 ISL 带宽。

3.4 StarCDN: 不可用性下的鲁棒性

StarCDN 还持续监控邻近卫星链路和桶 ID 的可达性。当前 ISL 可保持数周的高稳定性。然而, 卫星进行碰撞规避机动时链路不可用不可避免。此外, 缓存服务器不可用 (如软件更新时) 也较为常见但通常是暂时的。

StarCDN 对此类故障的响应取决于故障是瞬时性还是长期性的。对于瞬时性故障, StarCDN 简单地报告缓存未命中并将请求转发到地面。对于长期性故障, StarCDN 的一致性哈希方案将分配给不可用卫星的桶重新映射到下一个可用卫星 (该卫星现在负责多个桶)。我们的方法暂时导致负载不均衡, 但此类故障的持续时间足够短, 无需对一致性哈希方案进行大规模重新配置。我们在 §5.4 中评估了对故障鲁棒性。

SpaceGEN: 面向卫星 CDN 的合成流量生成器

本节描述我们的流量生成工具 SpaceGEN, 它能生成 LSN 用户内容访问的真实合成流量轨迹。我们的目标是从有限的真实流量中生成适用于 StarCDN 及其他卫星 CDN 设计仿真的长期合成轨迹。我们的工具基于足迹描述符 (Footprint Descriptors, FD) 理论构建。FD 是捕获用户内容访问方式的流量模型, 最初被提

出用于预测缓存命中率，并在 Akamai 生产 CDN 中用于缓存配置。与之前仅为单一位置的单个缓存生成合成轨迹的工具不同，SpaceGEN 能为模拟卫星 CDN 所需的多个位置集体生成合成轨迹。我们进一步展示，在对遍历这些位置的卫星进行缓存仿真时，合成轨迹和生产轨迹产生相似的命中率。我们已将该工具及从生产 CDN 推导的相关流量模型公开发布。

4.1 面向卫星 CDN 的流量模型

我们的模型使用以下流量模型来捕获 LSN 用户请求的对象的流行度、大小、重叠度和访问模式等关键统计特征：(i) 全局流行度分布（GPD），和 (ii) 流行度-大小足迹描述符（pFD）。

全局流行度分布（GPD）：GPD 捕获对象在多个位置的流行度和大小的联合分布。形式上，它表示为 $P(p_1, \dots, p_n, z)$ ，其中 p_i 表示对象在第 i 个位置的流行度， z 表示对象大小（以字节为单位）。流行度 p_i 定义为生产轨迹中对该对象在位置 i 的请求次数。

流行度-大小足迹描述符（pFD）：pFD 捕获来自单一位置的请求访问模式、流行度和对象大小。已证明其可捕获：(i) 对象级属性：包括流行度分布、大小分布和请求-大小分布；(ii) 缓存级属性：如请求命中率曲线和字节命中率曲线。形式上，pFD 描述为概率分布 $P(p, z, s, t)$ ，其中 p 和 z 分别表示对象在轨迹中的流行度和大小， s 表示对象连续访问之间请求的唯一字节数（称为栈距离）， t 表示连续访问之间的时间间隔。

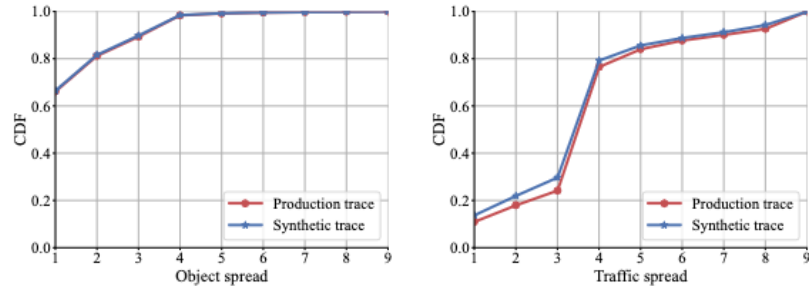
4.2 流量生成算法

SpaceGEN 使用的合成流量生成算法以 GPD 和从每个位置生产轨迹推导的 n 个 pFD 作为输入，生成 n 条用户指定长度的合成轨迹，每条对应一个特定位置。

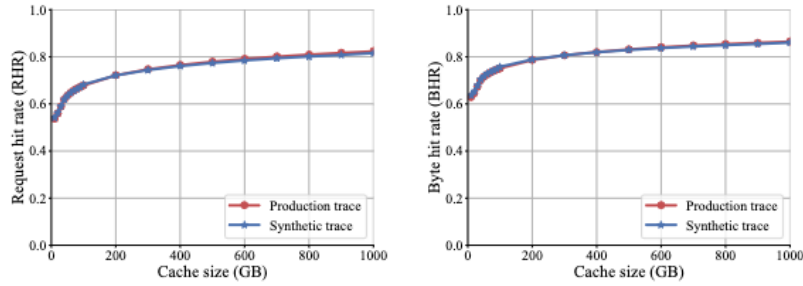
初始化阶段：我们为每个位置 i 初始化一个空缓存 C_i 。通过从 GPD 采样迭代创建对象，为其分配各位置的流行度和大小。对于每个对象 o ，如果其第 i 个位置的流行度 $p_i > 0$ ，则将 o 添加到 C_i ，直到 C_i 中对象大小之和达到第 i 个位置 pFD 的最大栈距离。

生成阶段：我们为 n 个位置生成 n 条合成轨迹。首先，对于每个位置 i ，计算 pFD 中所有可能的 p 和 z 值的栈距离分布 $P_i(s|p, z)$ 。在每次迭代中，检查缓存 C_i 顶部的对象。设该对象的流行度为 p 、大小为 z 。然后从 $P_i(s|p, z)$ 采样栈距离 s 。如果该对象已收到 p 次请求，则从合成轨迹中移除并重新插入 C_i 的栈距离 s 处。否则将其移除并重新插入。最后，根据 pFD 推导的平均数据率或真实轨迹计算的更精细数据率为轨迹分配时间戳。详细算法见附录 A.1。

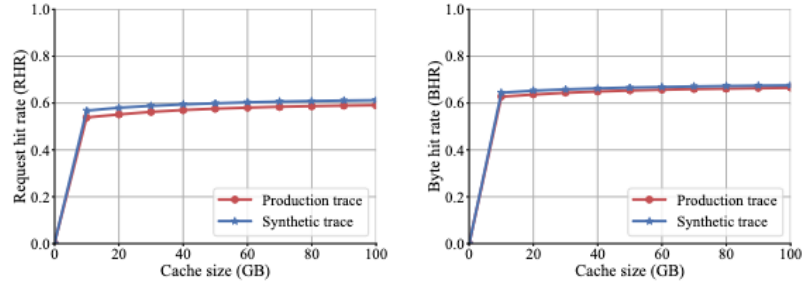
4.3 合成流量的特性



(a) Object spread across locations in the synthetic and production traces. (b) Traffic spread across locations in the synthetic and production traces.



(c) Request hit rates in a CDN LRU simulation for different cache sizes. (d) Byte hit rate in a CDN LRU simulation for different cache sizes.



(e) Request hit rate of a satellite equipped with an LRU cache for different cache sizes. (f) Byte hit rate of a satellite equipped with an LRU cache for different cache sizes.

Figure 6: Comparison of synthetic and production traces.

我们展示由算法 1 生成的合成轨迹与生产轨迹具有相似的属性。合成和生产轨迹的对象分布（object spread，即对象被访问的位置数量分布）和流量分布（traffic spread，即按大小和请求数量加权的对象分布）在图 6a 和图 6b 中呈现，两者高度相似。

在图 6c 和图 6d 中，我们展示了传统 CDN 服务器使用 LRU 的缓存仿真中，合成和生产轨迹产生相似请求命中率和字节命中率。具体而言，在所有缓存大小下，请求命中率的平均差异为 0.4%，字节命中率的平均差异为 0.3%。进一步地，当模拟运动中的卫星（配备 LRU 缓存）时，请求命中率的平均差异为 2%（字节命中率为 1%）。当模拟 StarCDN-Fetch 架构时也观察到类似结果。因此，我们得出结论：合成轨迹可以替代生产轨迹用于我们的评估。

实验评估

5.1 实验设置

CDN 流量轨迹：为在足够长的时间段内评估 StarCDN，我们使用 SpaceGEN 流量生成器 (§4) 生成了 5 天的合成轨迹。SpaceGEN 基于来自 Akamai 生产轨迹的流量模型为视频流量类别 (§3.1) 生成合成轨迹。视频流量类别的合成轨迹共包含 20 亿次请求和 2.5PB 内容。此外，我们在 §5.5 中扩展评估至 Web 和下载流量类别。

仿真设置：我们收集了 Starlink-53-Gen1 卫星的最新 TLE 数据（来自 Celestrak）和轨道壳信息（来自 Starlink.sx）。尽管 Gen1 卫星不支持 ISL，但 Starlink Gen2 星座仍处于发射阶段且尚未完全运行。因此，我们使用 Starlink-53-Gen1 作为 Starlink 拓扑的代表。我们通过壳信息推断轨道内和轨道间邻居的 ISL。我们在 72 条倾斜 53 度的轨道上模拟了 1170 颗卫星。

我们实现了一个流量驱动仿真器，由 Microsoft 的 CosmicBeats 仿真器（用于轨道运动、客户端链路调度）和多进程缓存重放器（用于模拟真实的异步 CDN 访问）组成。对于 CosmicBeats 中的每个 CDN 用户节点，我们关联一条来自 SpaceGEN 的合成轨迹。CosmicBeats 的时间步长设为 15 秒，与 Starlink 全局

调度器的重配置间隔一致。缓存重放器为每颗卫星生成一个使用 TCP 模拟 ISL 的进程。我们已开源该仿真框架。

基线方案：我们将 StarCDN 与两个基线进行比较：(a) 朴素 LRU——在 LEO 卫星上放置 LRU 缓存（如已有工作所提出）。(b) 静态缓存——一种理想化基线，假设没有轨道运动，卫星固定不动，用户-卫星映射为静态。该基线在实际中不可实现，但作为卫星 CDN 的性能上界。

5.2 缓存命中率

我们在图 7 中绘制了不同缓存大小下的命中率曲线。我们绘制了 $K=4$ 和 $K=9$ 两种哈希桶数的请求命中率（图 7a、7c）和字节命中率（图 7b、7d）。

如图所示，StarCDN 显著优于 LRU 基线。在缓存大小为 50GB、 $K=4$ 时，LRU 的请求命中率为 60%，而 StarCDN 达到 71%。最大命中率差距为 15%（缓存大小 60GB， $K=9$ ）。字节命中率方面也呈现类似趋势。StarCDN 能显著降低地面与太空之间的带宽利用率。

我们在图 8 中绘制了不同方案的上行带宽利用率（以无缓存的 Starlink 为基准进行归一化）。LRU 使用所有需从地面获取数据带宽的 30-35%，已展示了太空缓存的优势。StarCDN 更进一步，仅使用 20-25% 的上行带宽，为 LSN 运营商节省大量带宽。

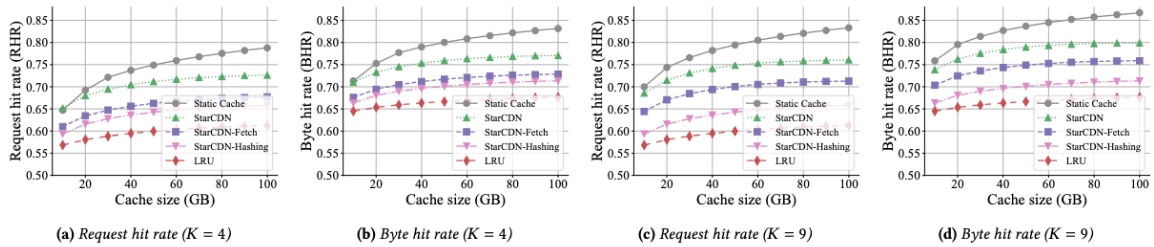


Figure 7: Request hit rate curve and byte hit rate curve for StarCDN variants, the LRU baseline, and the ideal static cache.

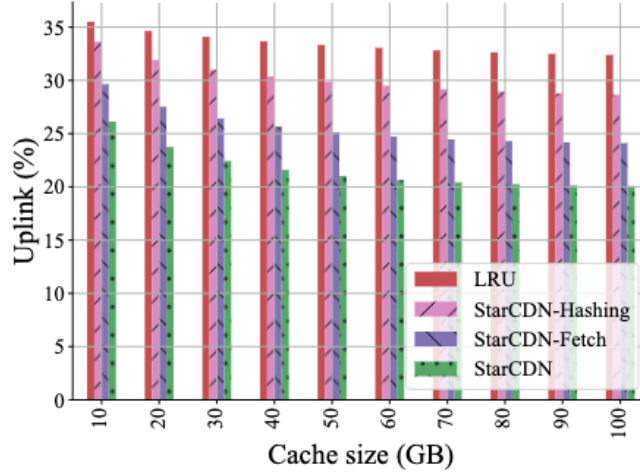


Figure 8: Uplink usage of StarCDN variants ($K = 9$) schemes and the LRU baseline, normalized with respect to Starlink with no cache.

5.2.1 一致性哈希的收益

我们评估一致性哈希对请求命中率和字节命中率的影响。与朴素 LRU 相比，StarCDN-Fetch（即含取回的 StarCDN）在请求命中率上平均提升 6 个百分点，字节命中率平均提升 4.8 个百分点（图 7a、7b）。当 $K=9$ 个哈希桶时，请求命中率（字节命中率）平均提升 9.7 个百分点（7.8 个百分点）。星间哈希和卫星缓存分区的一个直接效果是增加了太空中的有效缓存大小。然而，StarCDN 一致性哈希的收益超越了单纯增加缓存大小。考虑图 7c 中 $K=9$ 的情况，有效缓存大小增加了 9 倍。然而，LRU 在 90GB 时的请求命中率（61.3%）比 StarCDN 在 10GB 时的哈希命中率（64.4%）低 3.1 个百分点。这是因为相邻用户可能被调度到不同卫星，因此相同请求可能在不同缓存中处理。

5.2.2 中继取回的收益

中继取回在 StarCDN-Fetch 的基础上进一步带来 4.8 个百分点的请求命中率提升和 4.1 ($K=4$) / 3.9 ($K=9$) 个百分点的字节命中率提升。

在§3.3 中我们描述了中继取回如何利用历史流量足迹提升性能。为评估这一点，我们在表 3 中监控了缓存未命中时请求在轨道间邻居中的可用性。在 StarCDN 中，缓存未命中时请求被发送到最近的轨道间邻居副本。如果内容在邻居中可用，则从太空获取和提供。随着缓存大小增长，更多请求可仅由左侧邻居满足。这一收益的很大一部分来自卫星能够访问前序卫星的缓存。

Cache size (GB)	West only		East only		Both	
	Req	Byte	Req	Byte	Req	Byte
10	47.5	47.5	31.4	28.5	11.9	8.86
50	61.6	62.3	30.1	29.3	14.6	13.6
100	64.7	64.9	27.4	26.5	14.7	13.8

Table 3: Number of requests(Mil) and bytes(TB) available in inter-orbital neighbors with four hashing buckets during a miss

5.3 延迟

为评估 StarCDN 的延迟，我们估算了用户发起请求到请求从太空或地面 CDN 得到满足之间的端到端传播延迟。我们绘制了图 10 中的延迟 CDF。StarCDN 的中位延迟为 22ms，而无缓存的普通 Starlink 为 55ms，使我们更接近地面 CDN（服务地面用户）的性能水平。与地面 CDN 和 Starlink CDN 访问类似，StarCDN 由于缓存未命中也存在长尾延迟。当缓存未命中发生时，所有通过 ISL 的路由都成为开销，请求必须下行到地面网络。

哈希桶数与延迟之间存在权衡。由于网格形 ISL 的限制，哈希桶越多，路由到正确哈希桶所需的路由开销越大。图 9 展示了 K 增长时一致性哈希到正确桶的最坏情况路由延迟和请求命中率。当 K 超过 9 时，最坏情况延迟变得不可接受（40ms），即使额外命中率提升仅达 5%。当 K=9 时路由开销与 K=4 相同（因为 $2\lfloor \frac{\sqrt{K}}{2} \rfloor$ 对两种配置相同），但 K=9 在取回延迟方面可能产生额外开销。

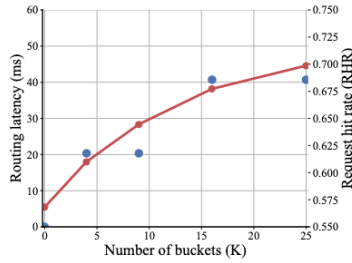
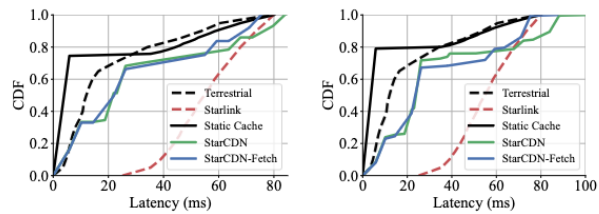


Figure 9: Worst case routing latency to the satellite hosting the correct hash bucket (points) and request hit rate (curve) with respect to number of buckets K. Both latency and hit rate increase with K.



(a) Latency CDF of StarCDN variants with K = 4 hash buckets, Terrestrial CDN, regular Starlink (no cache), and the Static Cache. **(b)** Latency CDF of StarCDN variants with K = 9 hash buckets, Terrestrial CDN, regular Starlink (no cache), and Static Cache.

Figure 10: Latency performance of StarCDN.

5.4 容错性

我们通过真实卫星星座状态评估 StarCDN 对不可用性的鲁棒性。收集轨道壳信息时，我们识别出 1296 颗活跃卫星中有 126 颗（9.7%）处于脱轨状态。这些不可

用卫星导致 438 条中断 ISL。请注意，StarCDN 会将每个请求哈希到正确的哈希桶。在服务正常的情况下，每个卫星节点将服务于一个哈希桶。然而，当 ISL 服务不可用时，由于 StarCDN 采用一致的哈希方案，一些卫星节点将被重新映射以服务于多个哈希桶。我们使用配备 50GB 缓存的九个哈希桶对 StarCDN 进行了实验。

我们在图 11 中绘制了服务于不同数量哈希 ID 的卫星节点的命中率。

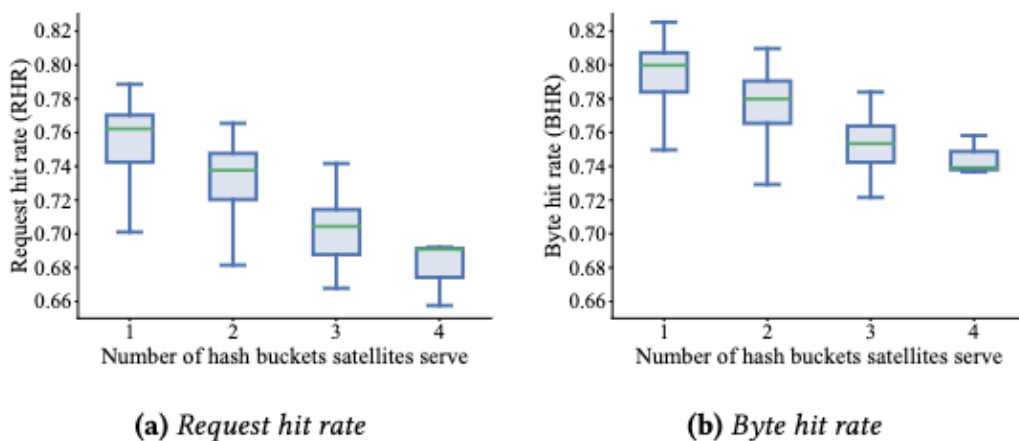


Figure 11: Hit rate performance of satellites serving different numbers of hash buckets.

当一颗卫星服务多个哈希 ID 时，请求命中率（字节命中率）最多下降 7 个百分点（5 个百分点）。虽然降幅明显，但与不使用 StarCDN 相比，StarCDN 仍能节省 74% 的上行带宽。

5.5 Web 和下载流量的评估

前述章节聚焦于视频内容分发。视频内容占 CDN 服务比特量的很大比例。然而，CDN 还服务多种其他流量类别，如网站和软件下载。不同流量类别具有不同的用户访问模式和对象大小分布，因此具有不同的缓存特性。

我们使用 SpaceGEN 生成了 5 天的合成轨迹：Web 合成轨迹包含 20 亿次请求、共 642TB 内容；下载轨迹包含 4.72 亿次请求、共 372TB 内容。相比视频轨迹，Web 和下载流量总量和内容足迹要小得多，因此命中率曲线随缓存大小增长更为平缓。

我们在图 12 中展示了 StarCDN 的命中率表现。对于两种流量类型，StarCDN 均显著优于 LRU 基线。特别是在图 12d 中，StarCDN 将字节命中率提升了超过 30%。此外，静态缓存如预期地提供了命中率的上限，且较少的缓存桶（ $K=4$ ）比更多桶（ $K=9$ ）提供更少的命中率。

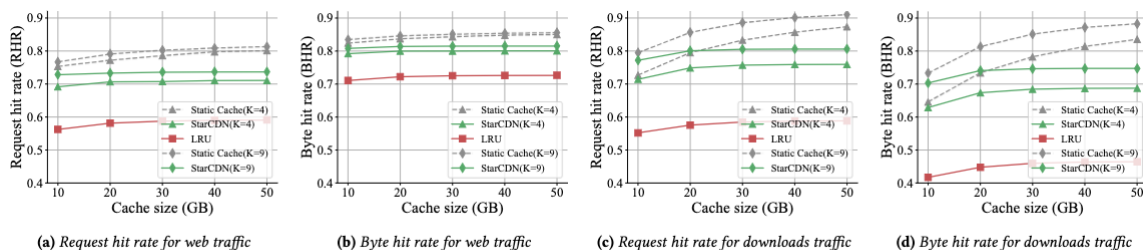


Figure 12: Request hit rate curve and byte hit rate curve for web and downloads traffic

相关工作

6.1 LEO 卫星网络

LEO 卫星网络在过去十年随着 Starlink、OneWeb 和 Kuiper 等服务的兴起受到了广泛研究。这些网络由于卫星速度、频繁的卫星切换以及天气引起的变化，本质上是动态的且带宽受限的。大量研究在网络栈的不同层面上测量和研究了性能优化。

我们的工作聚焦于更广泛的在轨计算领域。过去的工作首先提出了在卫星网络中进行在轨计算的思想，用于 CDN、卫星图像分析等多种应用。这些设计利用了小型 LEO 卫星中新兴的存储和计算能力。近期，一些研究进行了广泛的测量以识别地面 CDN 提供商在 LSN 场景下的不足（如§1 所述）。我们的工作在这一领域更深入地探索了 CDN 架构和设计。与以往工作不同，我们从全球多个位置收集真实流量轨迹，提出了专为卫星 CDN 设计的特定架构，设计了新技术，并使用真实流量轨迹进行了评估。此外，我们设计了一个合成流量生成器，可用于构建新的卫星缓存系统。

6.2 CDN 与缓存策略

缓存算法方面存在大量研究，基于流行度、对象大小、地理位置、时间、成本等多种因素做出缓存决策以提升缓存性能。CDN 的基础设施也得到了广泛研究。

Cloudflare、Google 和 Akamai 等领先企业已详细公开了其基础设施设计。我们

的工作与这些研究方向正交。我们专注于 LSN 中 CDN 的架构，设计了新的技术来应对轨道运动这一 LSN 独有的挑战。我们选择 LRU 作为驱逐算法，因其简单性和在实际部署中的广泛使用。除驱逐策略外，我们还专注于在 LSN 场景下独特有用的内容获取的关键方面。虽然我们的合成流量生成工作受到先前工作的启发，但我们是首个在合成轨迹中建模多位置间地理多样性的研究。

局限性与未来工作

本节讨论我们仿真框架和流量生成器中的一些局限性，以及流量生成和卫星 CDN 的未来工作方向。

- 跨位置时间流量相关性：我们的流量生成器和流量模型的一个局限性是未能完全捕获不同位置之间可能存在的微妙时间相关性。例如，CNN 上的热门新闻可能比在波士顿晚几个小时才在加利福尼亚流行起来。然而，如前所述，生产和合成轨迹之间的命中率差距非常小，表明这些相关性对缓存性能的影响有限。
- 离散时间仿真器：由于离散时间仿真器的固有局限性，我们当前的仿真框架无法建模对象传输过程中的断连。当我们在每颗卫星上重放缓存轨迹时，只会产生缓存命中或缓存未命中的结果，因此可能遗漏潜在的传输失败。Starlink 卫星约每五分钟触发一次切换，这可能导致传输失败。捕获这种行为需要更复杂的仿真器。我们将具有链路层仿真能力的更精细 LEO 卫星仿真器以及关键传输失败的影响留作未来工作。
- 星座-CDN 协同设计：在设计 StarCDN 时，我们基于公开信息建模了 Starlink 网络的现有部署。然而，一种替代设计可以联合优化星座和缓存策略，例如动态修改 ISL 拓扑或根据缓存未命中动态改变地面-卫星映射。我们未探索此类设计，因为星座设计受监管、可行性、发射成本等多种因素制约。因此，我们选择独立设计星座和缓存。
- 新型应用：LEO 卫星的轨道运动为协议栈的多个层面带来了挑战。随着卫星直连手机（direct-to-cell）服务的出现，我们预见在地理区域中维护用户状态——由于底层数据容器（卫星）不断移动——将成为卫星蜂窝服务的新挑战。

- 安全与 DNS: TLS 和相关加密技术对现代网络应用至关重要。边缘服务器托管用于终止 TLS 连接的密钥管理基础设施 (KMI) 的密钥。同样, 快速高效的 DNS 基础设施对于将客户端解析到首接触卫星在实际部署中也发挥着至关重要的作用。我们认为这些是未来工作的重要课题。
- CDN 与 LSN 的协同优化: 当前地面 CDN 与 StarCDN 之间的一种中间设计是将边缘服务器与地面站共置。虽然这种设计可以改善用户的 QoE, 但可能无法显著降低地面-卫星网络利用率和用户感知延迟。然而, 联合优化 LSN 和地面 CDN 的流量路由和内容缓存决策, 从性能和成本角度都值得探索。

结论

我们提出了 StarCDN, 一种新型天基 CDN 架构, 使部署在太空中的边缘服务器能够高效缓存内容。StarCDN 实现了在卫星缓存中放置和获取内容的机制, 以减少用户感知延迟并优化地面-卫星链路的利用率。我们还开发了首个开源合成流量生成器 SpaceGEN, 它可以从全球分布的用户群体中生成真实的内容访问合成轨迹。我们希望该工具和我们的数据集能够推动未来卫星 CDN 和缓存研究的发展。

伦理声明: 我们的工作使用了匿名化的生产 CDN 轨迹, 不涉及任何伦理问题。